```
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
```

```
import torch
torch.cuda.empty_cache()
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cuda
```

```
import os
!pip install datasets

from datasets import load_dataset

algebra_ds = load_dataset("EleutherAI/hendrycks_math", "algebra")
geometry_ds = load_dataset("EleutherAI/hendrycks_math", "geometry")
number_theory_ds = load_dataset("EleutherAI/hendrycks_math", "number_theory")
precalculus_ds = load_dataset("EleutherAI/hendrycks_math", "precalculus")

subjects = [
    "algebra",
    "counting_and_probability",
    "geometry",
    "intermediate_algebra",
    "number_theory",
    "prealgebra",
    "precalculus"
]

datasets_list = []

for subject in subjects:
    ds = load_dataset("EleutherAI/hendrycks_math", subject)
    datasets_list.append(ds["train"])

from datasets import concatenate_datasets

combined_ds = concatenate_datasets(datasets_list)
combined_ds = combined_ds.add_column("original_index", list(range(len(combined_ds))))

# Ensure the directory exists before saving and mounting drive later
# os.makedirs("/content/drive/MyDrive/math_datasets", exist_ok=True)

from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
combined_ds.save_to_disk("/content/drive/MyDrive/math_datasets/hendrycks_math_full")
```

```
print(combined_ds)
print(combined_ds[0])
```

PREPROCESSING

```
def preprocess(example):
    example["text"] = f"Problem:\n{example['problem']}\n\nSolution:\n{example['solution']}"
    return example

processed_ds = combined_ds.map(preprocess)
```

```
# imo_ds = load_dataset("TamasSimonds/imo-dataset")
```

```
# def preprocess_imo(example):
#     example["text"] = f"Problem:\n{example['prompt']}\n\nSolution:\n{example['completion']}"
#     return example

# imo_processed = imo_ds["train"].map(preprocess_imo)
```

```
processed_ds.save_to_disk("/content/drive/MyDrive/math_datasets/hendrycks_processed")
# imo_processed.save_to_disk("/content/drive/MyDrive/math_datasets/imo_processed")
```

```
split_ds = processed_ds.train_test_split(test_size=0.1)
train_ds = split_ds["train"]
test_ds = split_ds["test"]
```

LOAD MODEL

```
!pip install transformers peft accelerate bitsandbytes trl
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch

model_name = "Qwen/Qwen2.5-Math-7B"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4"
)
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
```

```
sample = test_ds[0]
problem = sample["problem"]

prompt = f"""Solve this olympiad math problem step by step.

Problem:
{problem}

Clearly explain your reasoning and state the final answer at the end.
"""
```

WITHOUT FINE TUNING

```
import re
import json
import torch
from datasets import load_from_disk
import os # Added import for os module

# ====================================================
# CREATE FIXED HENDRYCKS ALGEBRA BENCHMARK (if it doesn't exist)
# ====================================================
fixed_benchmark_path = "/content/drive/MyDrive/math_datasets/fixed_algebra_500"

if not os.path.exists(fixed_benchmark_path):
    print(f"Fixed algebra benchmark not found at {fixed_benchmark_path}. Creating it now...")
    # Load the processed dataset (assuming 'hendrycks_processed' exists from prior steps)
    loaded_processed_ds = load_from_disk("/content/drive/MyDrive/math_datasets/hendrycks_processed")

    # Filter for algebra problems
    algebra_ds_filtered = loaded_processed_ds.filter(lambda x: x["type"] == "Algebra")

    # Take the first 500 samples for a fixed benchmark
    # Ensure not to exceed available samples if less than 500
    fixed_algebra_benchmark = algebra_ds_filtered.select(range(min(500, len(algebra_ds_filtered))))

    # Save the fixed benchmark
    fixed_algebra_benchmark.save_to_disk(fixed_benchmark_path)
    print(f"Created fixed algebra benchmark with {len(fixed_algebra_benchmark)} questions.")
else:
```

```
    print(f"Fixed algebra benchmark found at {fixed_benchmark_path}. Loading existing dataset.")


# ======================================================
# LOAD FIXED HENDRYCKS ALGEBRA BENCHMARK
# ======================================================

fixed_algebra_test = load_from_disk(
    fixed_benchmark_path
)

print("Loaded fixed benchmark questions:", len(fixed_algebra_test))


# ======================================================
# ROBUST ANSWER EXTRACTION
# ======================================================

def extract_boxed_answer(text):

    # Try boxed answer first
    boxed = re.findall(r'\\boxed\{(.*?)\}', text)

    if boxed:
        return boxed[-1].strip()

    # Fallback to last number
    fallback = re.findall(r'[-+]?\d*\.?\d+', text)

    if fallback:
        return fallback[-1]

    # If no valid answer found
    return None


# ======================================================
# EVALUATION SETTINGS
# ======================================================

correct = 0
total_questions = len(fixed_algebra_test)

results = []


# ======================================================
# MAIN LOOP
# ======================================================

for i in range(total_questions):
```

```
    sample = fixed_algebra_test[i]
    problem = sample["problem"]

    prompt = f"""Solve this olympiad algebra problem step by step.

Problem:
{problem}

Clearly explain every reasoning step and state the final answer in \boxed{{}} format.
"""

    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    ).to(device)

    with torch.no_grad():

        outputs = model.generate(
            **inputs,
            max_new_tokens=256,
            do_sample=False
        )

    response = tokenizer.decode(
        outputs[0],
        skip_special_tokens=True
    )

    predicted_answer = extract_boxed_answer(response)
    actual_answer = extract_boxed_answer(sample["solution"])


    # ====================================================
    # STRICT ACCURACY LOGIC
    # ====================================================

    # If predicted answer is missing, auto-fail
    if predicted_answer is None:
        is_correct = False

    # If actual answer missing, also fail
    elif actual_answer is None:
        is_correct = False

    # Normal comparison
    else:
        is_correct = predicted_answer == actual_answer
```

```
if is_correct:
    correct += 1


# ======================================================
# SAVE RESULTS
# ======================================================

results.append({
    "question_id": i,
    "original_dataset_index": sample["original_index"],
    "category": sample["type"],
    "difficulty": sample["level"],
    "problem": sample["problem"],
    "generated_solution": response,
    "actual_solution": sample["solution"],
    "predicted_answer": predicted_answer if predicted_answer is not None else "NO ANSWER",
    "actual_answer": actual_answer if actual_answer is not None else "UNKNOWN",
    "correct": is_correct
})


# ======================================================
# DISPLAY RESULTS
# ======================================================

print("\n" + "=" * 120)
print(f"QUESTION {i+1}/{total_questions}")
print(f"Original Dataset Index : {sample['original_index']}")
print(f"Category               : {sample['type']}")
print(f"Difficulty Level       : {sample['level']}")
print("=" * 120)

print("\nALGEBRA PROBLEM:\n")
print(sample["problem"])

print("\nMODEL STEP-BY-STEP GENERATED SOLUTION:\n")
print(response)

print("\nACTUAL HENDRYCKS DATASET SOLUTION:\n")
print(sample["solution"])

print("\nFINAL ANSWER COMPARISON:")
print(f"Predicted Answer       : {predicted_answer if predicted_answer is not None else 'NO ANSWER'}")
print(f"Actual Answer          : {actual_answer if actual_answer is not None else 'UNKNOWN'}")
print(f"Correct                : {'YES' if is_correct else 'NO'}")

print("=" * 120 + "\n")
```

```
# ======================================================
# FINAL PERFORMANCE SUMMARY
# ======================================================

accuracy = (correct / total_questions) * 100

print("\nFINAL PERFORMANCE SUMMARY")
print("=" * 70)
print(f"Dataset Used           : Hendrycks MATH (Algebra Only)")
print(f"Total Questions Tested : {total_questions}")
print(f"Correct Answers        : {correct}")
print(f"Incorrect Answers      : {total_questions - correct}")
print(f"Final Accuracy         : {accuracy:.2f}%")
print("=" * 70)


# ======================================================
# SAVE DETAILED RESULTS
# ======================================================

with open(
    "/content/drive/MyDrive/math_datasets/baseline_results_detailed.json",
    "w"
) as f:
    json.dump(results, f, indent=4)

print("Detailed results saved successfully.")
```

```
Fixed algebra benchmark found at /content/drive/MyDrive/math_datasets/fixed_algebra_500. Loading existing dataset.
[transformers] Setting `pad_token_id` to `eos_token_id`:151643 for open-end generation.
Loaded fixed benchmark questions: 500
/usr/local/lib/python3.12/dist-packages/bitsandbytes/backends/cuda/ops.py:468: FutureWarning: _check_is_size will be removed in a future PyTorch release along with gua
  torch._check_is_size(blocksize)
[transformers] Setting `pad_token_id` to `eos_token_id`:151643 for open-end generation.

================================================================================================================================
QUESTION 1/500
Original Dataset Index : 0
Category               : Algebra
Difficulty Level       : Level 5
================================================================================================================================

ALGEBRA PROBLEM:

Let \[f(x) = \left\{
\begin{array}{cl} ax+3, &\text{ if }x>2, \\
x-5 &\text{ if } -2 \le x \le 2, \\
2x-b &\text{ if } x <-2.
\end{array}
\right.\]Find $a+b$ if the piecewise function is continuous (which means that its graph can be drawn without lifting your pencil from the paper).

MODEL STEP-BY-STEP GENERATED SOLUTION:
```

```
Solve this olympiad algebra problem step by step.

Problem:
Let \[f(x) = \left\{
\begin{array}{cl} ax+3, &\text{ if }x>2, \\
x-5 &\text{ if } -2 \le x \le 2, \\
2x-b &\text{ if } x <-2.
\end{array}
\right.\]Find $a+b$ if the piecewise function is continuous (which means that its graph can be drawn without lifting your pencil from the paper).

Clearly explain every reasoning step and state the final answer inoxed{} format.
To ensure the piecewise function \( f(x) \) is continuous, we need to check the continuity at the points where the definition of the function changes, i.e., at \( x =

1. **Continuity at \( x = 2 \):**
   - From the right (\( x > 2 \)): \( f(x) = ax + 3 \)
   - From the left (\( -2 \le x \le 2 \)): \( f(x) = x - 5 \)

   For continuity at \( x = 2 \):
   \[
   \lim_{x \to 2^+} f(x) = \lim_{x \to 2^-} f(x)
   \]
   \[
   a \cdot 2 + 3 = 2 - 5
   \]
   \[
   2a + 3 = -3
   \]
   \[
   2a = -6
   \]
   \[
   a = -3
```

```
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
```

FINE TUNING

```
from peft import LoraConfig, get_peft_model
lora_config = LoraConfig(
r=16,
lora_alpha=32,
target_modules=["q_proj", "v_proj"],
lora_dropout=0.05,
bias="none",
task_type="CAUSAL_LM"
)
```

```
model = get_peft_model(model, lora_config)

model.print_trainable_parameters()
```

```
trainable params: 5,046,272 || all params: 7,620,662,784 || trainable%: 0.0662
```

TOKENIZATION

```
def tokenize_function(example):
    tokens = tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=512
    )

    tokens["labels"] = tokens["input_ids"].copy()

    return tokens
```

```
train_tokenized = train_ds.map(tokenize_function, batched=False)
test_tokenized = test_ds.map(tokenize_function, batched=False)
```

TRAIN THE MODEL

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="/content/drive/MyDrive/qwen_math_finetuned",
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=4,
    num_train_epochs=5,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_steps=50,
    learning_rate=2e-5,
    fp16=True,
    report_to="none",
    remove_unused_columns=False
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized,
    eval_dataset=test_tokenized
```

```
)

trainer.train()
```

```
trainer.save_model("/content/drive/MyDrive/qwen_math_final")
tokenizer.save_pretrained("/content/drive/MyDrive/qwen_math_final")
```

```
!pip install sympy
from sympy import simplify, sympify

def math_equal(pred, actual):
    try:
        return simplify(sympify(pred) - sympify(actual)) == 0
    except:
        return str(pred).strip() == str(actual).strip()
```

```
import re
import json
import torch
from datasets import load_from_disk
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from peft import PeftModel
import os

# ====================================================
# DEVICE SETUP
# ====================================================

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

# ====================================================
# LOAD BASE MODEL + FINE-TUNED LORA MODEL
# ====================================================

model_name = "Qwen/Qwen2.5-Math-7B"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4"
)

tokenizer = AutoTokenizer.from_pretrained(model_name)

# Load base quantized model
base_model = AutoModelForCausalLM.from_pretrained(
    model_name,
```

```
    quantization_config=bnb_config,
    device_map="auto"
)

# Load your fine-tuned LoRA checkpoint
model = PeftModel.from_pretrained(
    base_model,
    "/content/drive/MyDrive/qwen_math_final"
)

model.eval()

# ======================================================
# CREATE FIXED HENDRYCKS ALGEBRA BENCHMARK (if not exist)
# ======================================================

fixed_benchmark_path = "/content/drive/MyDrive/math_datasets/fixed_algebra_500"

if not os.path.exists(fixed_benchmark_path):
    print(f"Fixed algebra benchmark not found at {fixed_benchmark_path}. Creating it now...")

    loaded_processed_ds = load_from_disk(
        "/content/drive/MyDrive/math_datasets/hendrycks_processed"
    )

    algebra_ds_filtered = loaded_processed_ds.filter(
        lambda x: x["type"] == "Algebra"
    )

    fixed_algebra_benchmark = algebra_ds_filtered.select(
        range(min(500, len(algebra_ds_filtered)))
    )

    fixed_algebra_benchmark.save_to_disk(fixed_benchmark_path)

    print(
        f"Created fixed algebra benchmark with {len(fixed_algebra_benchmark)} questions."
    )

else:
    print(
        f"Fixed algebra benchmark found at {fixed_benchmark_path}. Loading existing dataset."
    )

# ======================================================
# LOAD FIXED BENCHMARK
# ======================================================

fixed_algebra_test = load_from_disk(
    fixed_benchmark_path
)
```

```
print("Loaded fixed benchmark questions:", len(fixed_algebra_test))

# ====================================================
# ROBUST ANSWER EXTRACTION
# ====================================================

def extract_boxed_answer(text):

    boxed = re.findall(r'\\boxed\{(.*?)\}', text)

    if boxed:
        return boxed[-1].strip()

    fallback = re.findall(r'[-+]?\d*\.?\d+', text)

    if fallback:
        return fallback[-1]

    return None


# ====================================================
# EVALUATION SETTINGS
# ====================================================

correct = 0
total_questions = len(fixed_algebra_test)

results = []

# ====================================================
# MAIN LOOP
# ====================================================

for i in range(total_questions):

    sample = fixed_algebra_test[i]
    problem = sample["problem"]

    prompt = f"""Solve this olympiad algebra problem step by step.

Problem:
{problem}

Clearly explain every reasoning step and state the final answer in \\boxed{{}} format.
"""

    inputs = tokenizer(
        prompt,
        return_tensors="pt",
```

```
            truncation=True,
            max_length=512
        ).to(device)

        with torch.no_grad():

            outputs = model.generate(
                **inputs,
                max_new_tokens=256,
                do_sample=False
            )

        response = tokenizer.decode(
            outputs[0],
            skip_special_tokens=True
        )

        predicted_answer = extract_boxed_answer(response)
        actual_answer = extract_boxed_answer(sample["solution"])

        # ====================================================
        # FLEXIBLE MATHEMATICAL ACCURACY LOGIC
        # ====================================================

    if predicted_answer is None:
        is_correct = False

    elif actual_answer is None:
        is_correct = False

    else:
        is_correct = math_equal(
            predicted_answer,
            actual_answer
        )

    if is_correct:
        correct += 1
        # ====================================================
        # SAVE RESULTS
        # ====================================================

        results.append({
            "question_id": i,
            "original_dataset_index": sample["original_index"],
            "category": sample["type"],
            "difficulty": sample["level"],
            "problem": sample["problem"],
            "generated_solution": response,
            "actual_solution": sample["solution"],
            "predicted_answer": predicted_answer if predicted_answer is not None else "NO ANSWER",
```

```
            "actual_answer": actual_answer if actual_answer is not None else "UNKNOWN",
            "correct": is_correct
        })

        # ======================================================
        # DISPLAY RESULTS
        # ======================================================

        print("\n" + "=" * 120)
        print(f"QUESTION {i+1}/{total_questions}")
        print(f"Original Dataset Index : {sample['original_index']}")
        print(f"Category               : {sample['type']}")
        print(f"Difficulty Level       : {sample['level']}")
        print("=" * 120)

        print("\nALGEBRA PROBLEM:\n")
        print(sample["problem"])

        print("\nFINE-TUNED MODEL STEP-BY-STEP GENERATED SOLUTION:\n")
        print(response)

        print("\nACTUAL HENDRYCKS DATASET SOLUTION:\n")
        print(sample["solution"])

        print("\nFINAL ANSWER COMPARISON:")
        print(
            f"Predicted Answer       : {predicted_answer if predicted_answer is not None else 'NO ANSWER'}"
        )
        print(
            f"Actual Answer          : {actual_answer if actual_answer is not None else 'UNKNOWN'}"
        )
        print(f"Correct                : {'YES' if is_correct else 'NO'}")

        print("=" * 120 + "\n")

    # ======================================================
    # FINAL PERFORMANCE SUMMARY
    # ======================================================

    accuracy = (correct / total_questions) * 100

    print("\nFINAL PERFORMANCE SUMMARY")
    print("=" * 70)
    print(f"Dataset Used           : Hendrycks MATH (Algebra Only)")
    print(f"Model Type             : Fine-Tuned Qwen2.5-Math + LoRA")
    print(f"Total Questions Tested : {total_questions}")
    print(f"Correct Answers        : {correct}")
    print(f"Incorrect Answers      : {total_questions - correct}")
    print(f"Final Accuracy         : {accuracy:.2f}%")
    print("=" * 70)
```

```
# ======================================================
# SAVE DETAILED RESULTS
# ======================================================

with open(
    "/content/drive/MyDrive/math_datasets/fine_tuned_results_detailed.json",
    "w"
) as f:
    json.dump(results, f, indent=4)

print("Fine-tuned detailed results saved successfully.")
```